

CampusQuant AI

Data-Driven Talent Forecasting and Placement Intelligence System

Technical Documentation

1. Overview

CampusQuant AI is a centralized, role-based web application that brings students, the placement office, and recruiters onto a single platform. It replaces manual, record-keeping-only placement systems with a system that actively analyses student data, predicts placement likelihood, and assists every stakeholder using AI.

Two distinct forms of intelligence are used, kept deliberately separate:

- A trained machine learning model that predicts a student's placement likelihood from academic and activity data — explainable and consistent, since it depends on a fixed, versioned model.
- The Grok API (xAI), used wherever the system needs to reason over free text — resumes, chat queries, job descriptions — for feedback, mentoring, and summarisation.

2. System Architecture

2.1 Presentation Layer

A React (Vite) single-page application styled with Tailwind CSS. All three portals — Student, Placement Officer/Admin, and Recruiter — are served from the same app, with visible modules controlled by the logged-in user's role.

2.2 Application Layer

A FastAPI backend exposes REST endpoints for authentication, profiles, job postings, applications, and both intelligence layers. JWT tokens carry the user's role and are checked on every request to enforce access boundaries.

2.3 Data Layer

A MySQL database stores all structured data. Uploaded resumes and generated reports are stored directly on the server's own filesystem — no third-party storage account is required.

2.4 Intelligence Layer

Two independent components, both called only from the backend, never from the frontend directly: the trained classification model (Section 4) and the Grok API integration (Section 5). This separation means the model can be retrained, or the AI prompts refined, without touching the rest of the system.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, Tailwind CSS	Single-page application, role-based UI
UI Components	shadcn/ui, Recharts	Form components and analytics charts
Backend	FastAPI (Python)	REST API, request validation, auth
Database	MySQL	Relational storage for all structured data

ORM / Migrations	SQLAlchemy, Alembic	Database access and schema versioning
Authentication	JWT, bcrypt (passlib)	Stateless auth, password hashing
File Storage	Server local filesystem	Resume and report file storage
ML Framework	scikit-learn / XGBoost	Placement-likelihood classification model
LLM Integration	Grok API (xAI)	Resume feedback, mentoring, summarisation, recommendations
Resume Parsing	pdfplumber	Text extraction from uploaded PDF resumes
Notifications	SMTP (smtplib)	Email notifications and OTP delivery

4. Machine Learning Module

Placement-likelihood prediction is a supervised binary classification task, trained separately from and independent of the Grok integration.

4.1 Dataset

Trained on a placement-prediction dataset of 10,000 student records with no missing values, covering: CGPA, major and mini projects, workshops/certifications, a self-reported skills count, communication skill rating, internship experience, hackathon participation, 12th and 10th percentage, active backlogs, and the target label — Placement Status (Placed / Not Placed).

4.2 Preprocessing

- Binary encoding for categorical fields (Internship, Hackathon)
- Feature scaling where required by the chosen algorithm
- Stratified 80/20 train-test split to preserve class balance

4.3 Model Training

A Logistic Regression baseline establishes an interpretable reference point. A Random Forest / XGBoost classifier is then trained on the same split, typically improving on the baseline by capturing non-linear interactions between features such as CGPA, project count, and communication rating. The better-performing model is saved as a versioned artefact and loaded by the backend at request time.

4.4 Evaluation

Assessed on the held-out test split using accuracy, precision, recall, and F1-score, with particular attention to recall on the “Placed” class.

4.5 Serving Predictions

At inference time, the backend passes a student's current profile fields to the loaded model, which returns a placement probability. This is stored and surfaced to the student as a readiness indicator, and to the placement officer in aggregate form across a batch.

5. AI Integration (Grok API)

All Grok calls are made server-side through one reusable function, so the API key never reaches the frontend and usage can be rate-limited per user.

5.1 Student-Facing Features

- Resume feedback — extracted resume text sent to Grok for structured critique of content and impact
- Skill-gap analysis — student skills vs. a target job's requirements, compared by Grok
- AI mentor chatbot — free-text queries answered directly by Grok, with basic profile context
- Mock interview practice — Grok generates questions and scores typed responses
- Role recommendation — student profile and open postings passed together, ranked and explained by Grok

5.2 Placement Officer-Facing Features

- Batch summary generation — SQL-computed statistics handed to Grok for a plain-language report
- Skill-demand overview — skills counted across active postings, summarised into a trend narrative

5.3 Recruiter-Facing Features

- Candidate fit summaries — applicant profile + JD sent to Grok for a short fit assessment
- Interview question generation — role-specific questions generated from the JD
- Interview note structuring — recruiter's brief notes expanded into a formal writeup

6. Authentication

Students log in with a college-issued USN (e.g. 1CR23CD001), generated by the admin at account-creation time, never self-assigned. Uniqueness is enforced at the database level. Passwords are hashed with bcrypt; login issues a short-lived JWT access token plus a longer-lived refresh token, with the user's role embedded in the token and checked on every request.

7. Module-wise Functional Description

7.1 Student Module

Profile and resume management, ML-predicted placement likelihood with an AI-generated explanation, skill-gap review, AI mentor chat, and AI-ranked role recommendations.

7.2 Placement Officer / Admin Module

Student account and USN creation, eligibility rule configuration, batch-level dashboards of readiness and predicted placement rate, and one-click AI-generated drive reports.

7.3 Recruiter Module

Job posting and eligibility criteria, AI-ranked and summarised applicant lists, AI-generated interview questions, and structured interview feedback recording.

8. Deployment

The entire stack is self-hostable on a personal machine or the institution's own server — React frontend, FastAPI backend, and MySQL database all run locally with no paid third-party infrastructure. The Grok API is the only external service used.